Lecture 14

Exceptions

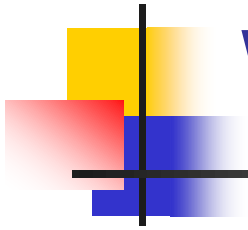
Handling Errors with Exceptions

Attack of the Exception

```
public static int average(int[] a) {
      int total = 0;
      for(int i = 0; i < a.length; i++) {
            total += a[i];
      }
      return total / a.length;
}
```

- What happens when this method is used to take the average of an array of length zero?
- Program throws an *Exception* and fails

```
java.lang.ArithmeticException: / by zero
```



What is an Exception?

- An error event that disrupts the program flow and may cause a program to fail.

- Some examples:
  - Performing illegal arithmetic
  - Illegal arguments to methods
  - Accessing an out-of-bounds array element
  - Hardware failures
  - Writing to a read-only file

Another Exception Example

- What is the output of this program?

```
public class ExceptionExample {
    public static void main(String args[]) {
        String[] greek = {"Alpha", "Beta"};
        System.out.println(greek[2]);
    }
}
```

Output:

```
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: 2
        at ExceptionExample.main(ExceptionExample.java:4)
```

Exception Message Details

Exception message format:

```
[exception class]: [additional description of exception]
      at [class].[method]([file]:[line number])
```

Example:

```
java.lang.ArrayIndexOutOfBoundsException: 2
        at ExceptionExample.main(ExceptionExample.java:4)
```

- What exception class? **ArrayIndexOutOfBoundsException**
- Which array index is out of bounds? **2**
- What method throws the exception? **main**
- What file contains the method? **ExceptionExample.java**
- What line of the file throws the exception? **4**

Exception Handling

- Use a `try-catch` block to handle exceptions that are thrown

```
try {
   // code that might throw exception
}
catch ([Type of Exception] e) {
   // what to do if exception is thrown
}
```

Exception Handling Example

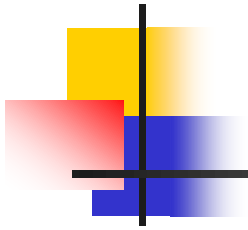
```
public static int average(int[] a) {
        int total = 0;
        for(int i = 0; i < a.length; i++) {
                total += a[i];
        }
        return total / a.length;
}

public static void printAverage(int[] a) {
        try {
                int avg = average(a);
                System.out.println("the average is: " + avg);
        }
        catch (ArithmeticException e) {
                System.out.println("error calculating average");
        }
}
```

Catching Multiple Exceptions

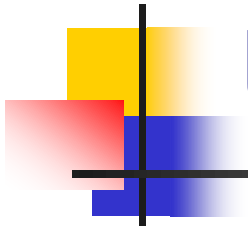
- Handle multiple possible exceptions by multiple successive catch blocks

```
try {
    // code that might throw multiple exception
}
catch (IOException e) {
    // handle IOException and all subclasses
}
catch (ClassNotFoundException e2) {
    // handle ClassNotFoundException
}
```



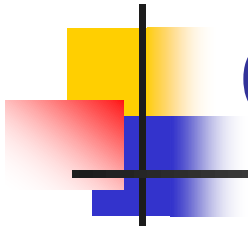
Exceptions Terminology

- When an exception happens we say it was **thrown** or **raised**
- When an exception is dealt with, we say the exception is was **handled** or **caught**



Unchecked Exceptions

- All the exceptions we've seen so far have been *Unchecked Exceptions*, or *Runtime Exceptions*
- Usually occur because of programming errors, when code is not robust enough to prevent them
- They are numerous and can be ignored by the programmer



Common Unchecked Exceptions

- `NullPointerException`

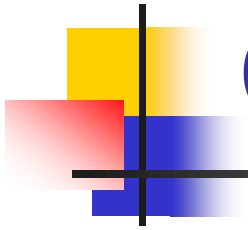
  reference is null and should not be

- `IllegalArgumentException`

  method argument is improper is some way

- `IllegalStateException`

  method called when class is in improper state



Checked Exceptions

- There are also *Checked Exceptions*
- Usually occur because of errors programmer cannot control:

  examples: hardware failures, unreadable files
- They are less frequent and they cannot be ignored by the programmer . . .

Dealing With Checked Exceptions

- Every method must catch (handle) checked exceptions or specify that it may throw them
- Specify with the **throws** keyword

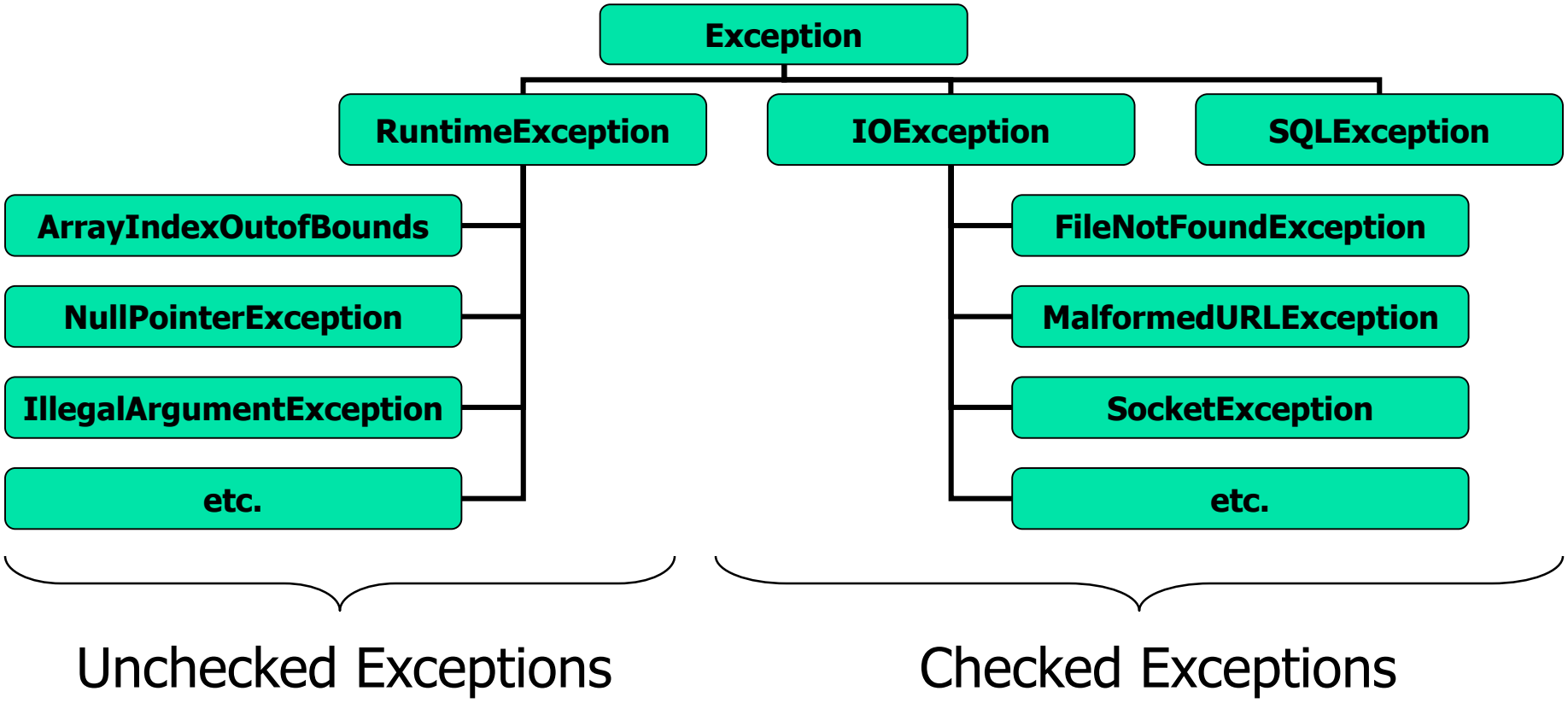
```
void readFile(String filename) {
    try {
        FileReader reader = new FileReader("myfile.txt");
        // read from file . . .
    } catch (FileNotFoundException e) {
        System.out.println("file was not found");
    }
}
```

or

```
void readFile(String filename) throws FileNotFoundException {
    FileReader reader = new FileReader("myfile.txt");
    // read from file . . .
}
```

Exception Class Hierarchy

- All exceptions are instances of classes that are subclasses of `Exception`



Checked and Unchecked Exceptions

Checked Exception	**Unchecked Exception**
not subclass of RuntimeException	subclass of RuntimeException
if not caught, method *must* specify it to be thrown	if not caught, method *may* specify it to be thrown
for errors that the programmer *cannot* directly prevent from occurring	for errors that the programmer *can* directly prevent from occurring,
IOException, FileNotFoundException, SocketException	NullPointerException, IllegalArgumentException, IllegalStateException

Exception Constructors

- Exceptions have at least two constructors:

1. no arguments

```
NullPointerException e = new NullPointerException();
```

2. single `String` argument

   descriptive message that appears when exception error message is printed

```
IllegalArgumentExceptione e =
  new IllegalArgumentException("number must be positive");
```

Writing Your Own Exception

- To write your own exception, write a subclass of `Exception` and write both types of constructors

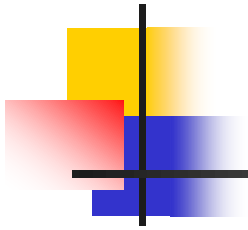
```
public class MyCheckedException extends IOException {
    public MyCheckedException() {}
    public MyCheckedException(String m) {super(m);}
}


public class MyUncheckedException extends RuntimeException {
    public MyUncheckedException() {}
    public MyUncheckedException(String m) {super(m);}
}
```

Throwing Exceptions

- Throw exception with the **throw** keyword

```
public static int average(int[] a) {
  if (a.length == 0) {
    throw new IllegalArgumentException("array is empty");
  }
  int total = 0;
  for(int i = 0; i < a.length; i++) {
    total += a[i];
  }
  return total / a.length;
}
```



Keyword Summary

- Four new Java keywords
- **`try`** and **`catch`** – used to handle exceptions that may be thrown
- **`throws`** – to specify which exceptions a method throws in method declaration
- **`throw`** – to throw an exception

Throws and Inheritance

- A method can throw less exceptions, but not more, than the method it is overriding

```
public class MyClass {
   public void doSomething()
             throws IOException, SQLException {
       // do something here
   }
}

public class MySubclass extends MyClass {
   public void doSomething() throws IOException {
       // do something here
   }
}
```

Line Intersection Example

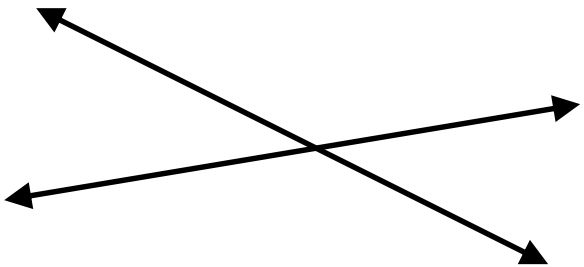
- Consider this class `Line`, which has two fields, a `slope` and a `yIntercept`

```
class Line {
  private double slope;
  private double yIntercept;
  double getSlope() { return slope; }
  double getYIntercept() { return yIntercept; }
}
```

- Let's write an intersect method that returns the x-coordinate at which the two lines intersect.

Boring Math Stuff . . .

- Calculating the x-coordinate at which two lines intersect



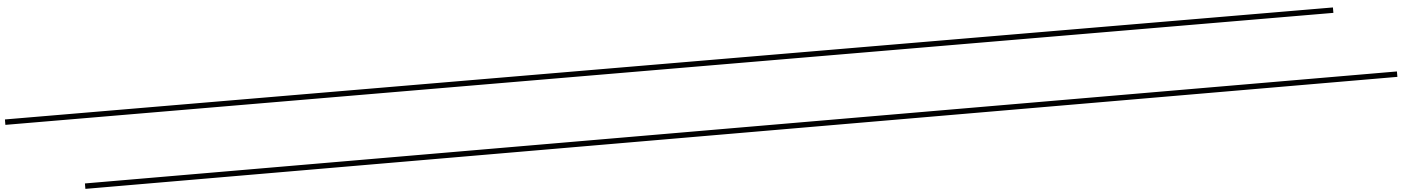
$$y = m_1x + b_1$$
$$y = m_2x + b_2$$
$$m_1x + b_1 = m_2x + b_2$$
$$m_1x - m_2x = b_2 - b_1$$
$$(m_1 - m_2)x = b_2 - b_1$$
$$\mathbf{x = (b_2 - b_1)/(m_1 - m_2)}$$

- We could translate this directly into the following intersect method:

```
double intersect(Line line1, Line line2) {
    return (line2.getYIntercept() - line1.getYIntercept()) /
           (line1.getSlope() - line2.getSlope())
}
```

What About Parallel Lines?

- Parallel lines will never intersect.



- If lines are parallel, then their slopes will be equal, `line1.slope() - line2.slope() = 0`, and our method will attempt to divide by zero

- Let's write a new exception `ParallelException` to throw when this occurs.

ParallelException

- `ParallelException` will be a checked exception because calculating whether lines are parallel is not something we expect the programmer to know how to do and prevent in advance.

- Checked exceptions are a subclass of `Exception`, but not `RuntimeException`

```
public class ParallelException extends Exception {
  public ParallelException() {}
  public ParallelException(String msg) { super(msg); }
}
```

Final Intersect Method

- Because it is a checked exception, `intersect` must specify that it throws it

```
double intersect(Line line1, Line line2)
        throws ParallelException
{
    if (line1.slope() == line2.slope()) {
        throw new ParallelException();
    }
    return (line2.getYIntercept() -
                    line1.yIntercept()) /
            (line1.slope() - line2.slope())
}
```

Calling the `intersect` Method

- A method that accepts two `Lines` as arguments, calls `intersect`, and prints out the results:

```
void printIntersect(Line line1, Line line2) {
    try {
        double x = intersect(line1, line2);
        System.out.println("Intersect at " + x);
    } catch (ParallelException e) {
        System.out.println("They are parallel");
    }
}
```